

Faithful, Sufficient and Understandable: Rethinking Graph Counterfactual Explanations via Discrete Diffusion Inversion

David Bechtoldt^{a,b} , Sidney Bender^{a,b} 

^a*Machine Learning Group, TU Berlin, 10587 Berlin, Germany*

^b*BIFOLD–Berlin Institute for the Foundations of Learning and Data, Berlin, Germany*

Abstract

Graph Neural Networks (GNNs) achieve strong predictive performance on graph-structured data across domains such as chemistry, biology, and network analysis, yet they provide no intrinsic explanation of their predictions. This limits their adoption in high-stakes and safety-critical settings. Counterfactual explanations address this by revealing the minimal structural modifications that would change a model’s prediction. On graphs, however, such a modification is hard to produce. The search space is discrete and combinatorial, and a valid answer must respect categorical node and edge types together with domain rules such as chemical valency in the case of molecular graphs. Existing explainers give up one of two things. Either edits are not held on the data manifold, or the search does not span the full edit space. We propose Graph Diffusion Counterfactual Explanation via Inversion (GDCE-I), which gives up neither. A discrete denoising diffusion model with a novel discrete inversion scheme enables distribution-aware edits leveraging the whole domain edit space. We further address the incomplete and inconsistent evaluation of graph counterfactuals by deriving a framework of explanation desiderata and applying it to every method under one shared protocol. Across four benchmarks, GDCE-I outperforms related work by a large margin on the defined framework. For the molecular domain, we further qualitatively show that GDCE-I attains interpretable in-distribution solutions.

Keywords: Graph Neural Networks, Counterfactual Explanations, Diffusion Models, Molecular Graphs

1. Introduction

Graph Neural Networks [1] have emerged as the de facto standard for learning graph-structured data across domains such as chemistry, biology, and network analysis, achieving breakthroughs in tasks such as molecular property prediction, drug discovery, and toxicity analysis [2, 3, 4]. Despite their predictive abilities, GNNs operate predominantly as black-box models. This lack of transparency undermines the decision-making process, severely limiting its adoption in high-stakes scenarios where interpretability and safety are paramount [5, 6, 7]. For example, the discovery of a new structural class of antibiotics was guided not by the prediction of twelve million candidates by a graph neural network, but by the chemical substructures the model had associated with selective activity [8].

To uncover such explanations, Explainable AI (XAI) offers a diverse set of approaches to explain a neural network. A prominent branch of XAI is attribution-based methods, such as Layer-wise Relevance Propagation [9] that was further developed for GNNs (GNN-LRP) [10]. These techniques answer the "why" behind a model's decision by identifying and highlighting the most influential features or structural motifs.

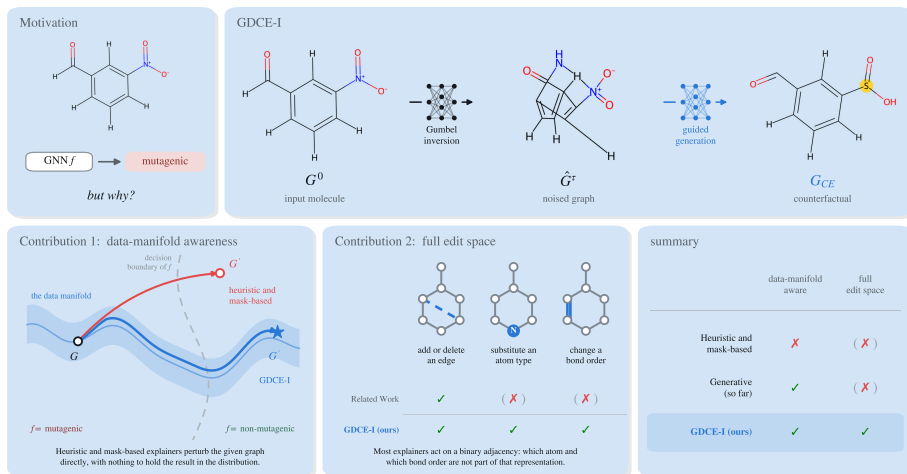


Figure 1: GDCE-I generates graph counterfactuals by inverting a discrete diffusion trajectory. Respecting the data manifold and covering the full edit space is what makes the explanations domain-valid.

Another approach to explaining a model is counterfactual explanations [11, 12, 13]. They provide a realistic counterexample for a given input, which differs as little as possible to alter the classifier's prediction. Rather than attributing relevance to existing structures, counterfactuals answer: "What minimal changes to the input graph would alter the model's prediction?" In computational chemistry, this could translate to targeted structural edits that can neutralize a toxicophore.

However, translating counterfactual generation to the graph domain introduces unique challenges that are largely absent in continuous domains such as computer vision. Graphs are inherently discrete, non-euclidean, and combinatorial objects [13, 14]. Crucially, real-world graph representations, such as molecules or proteins, reside in a highly sparse topological space [15]. Consequently, even minor topological perturbations can easily violate strict domain-specific rules, such as chemical valency, resulting in structurally invalid and semantically meaningless instances.

Early approaches relied on heuristic perturbation strategies restricted solely to edge deletions [13, 16]. This constrains the counterfactual search space, prohibiting the addition of necessary structural motifs or the alteration of edge and node types. To improve the realism of the explanations, a parallel line of work turns to generative models that keep counterfactuals close to the data distribution [14, 17, 18]. Although more recent heuristic methods support edge additions and node-type changes [19], most methods in both families still operate on a binary adjacency matrix that cannot represent categorical edge types. To systematically address these limitations, we propose *Graph Diffusion Counterfactual Explanation via Inversion* (GDCE-I)¹. By combining discrete denoising diffusion models [15] with classifier-free guidance [21, 22] and an edit-friendly inversion technique, our approach allows for comprehensive modifications, including targeted edge additions, deletions, edge and node type alterations, within a data-manifold-aware generation process (Figure 1).

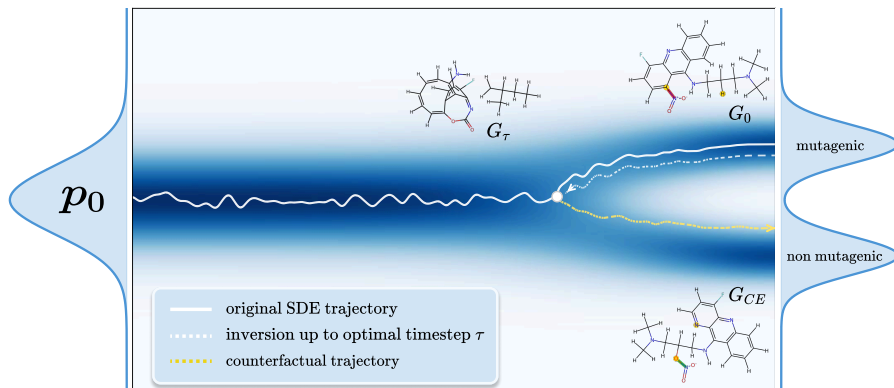


Figure 2: High-level illustration of GDCE-I. By manipulating the latent space of the learned diffusion model, we can generate distribution-aware and sparse counterfactuals.

GDCE-I integrates discrete diffusion models with classifier-free guidance, so that the model captures the conditional distribution of valid graphs directly. Its core contribution is an *edit-friendly* inversion scheme for discrete diffusion,

¹A preliminary version of this work was presented at the ESANN 2026 Conference [20]. This paper presents the fully extended framework.

an inversion that records the sampling stochasticity of a given graph, so the graph can be reconstructed and then minimally edited by changing only the target condition. We derive it by exploiting the Gumbel-Max trick in a discrete diffusion setting. We traverse a reference trajectory of the given graph in the reverse direction and, at every step, construct the Gumbel noise that forces the guided reverse posterior to reproduce that trajectory, using a truncated-Gumbel (A*-sampling) construction [23]. The resulting posterior noise space reconstructs the original graph exactly under its own condition. Re-injecting this same noise during a reverse generation pass guided toward a new target property therefore preserves the structural identity of the original graph as far as the new target permits, favoring edits that are both minimal and in distribution. The trade-off between sparsity and the desired property flip is controlled via a dynamic skip-parameter τ . A high-level illustration can be contemplated in Figure 2. Our main contributions are summarized as follows:

- **A Novel Generative Inversion Framework:** We introduce GDCE-I, the first method to unify discrete graph diffusion, classifier-free guidance, and Gumbel-Max inversion. This framework enables precise, edit-friendly manipulation of discrete graph structures.
- **Benchmarking on Classifier-Distilled Datasets:** Through a rigorous evaluation of the Mutagenicity [24], Benzene [25, 26], PROTEINS [27, 28] and TWITTER [29, 30] dataset against baselines such as CF^2 [31], C2Explainer [19], XPlore [32], UCExplainer [33], and D4Explainer [17], we show that GDCE-I is the only method that jointly produces faithful and understandable counterfactuals. We further provide qualitative evidence that GDCE-I recognizes and alters functional groups via realistic chemical substitutions for Mutagenicity and Benzene.

2. Related Work

Counterfactual Explanations. Although there are counterfactual explanation methods for domains such as natural language [34] and proteins [35], the most established fields are tabular data and computer vision. Tabular data often is low-dimensional and specific properties of the input features are known, so there exist a lot of approaches directly using this knowledge, e.g., in the form of Integer Linear Programming [36] or other explicit closed-form optimization schemes such as in DiCE [37]. For both tabular data and computer vision, there are various approaches that use a generative model to filter gradients to stay in the data manifold while moving towards the desired class [12, 38, 39, 40, 41, 42, 43, 44]. These approaches cannot be easily transferred to graphs due to the discrete nature. In addition, there are approaches for both tabular and computer vision in the form of distillation [45, 46]. These approaches are in principal transferable to graphs, but one has to choose a suitable generative model.

Heuristic and mask-based methods for Graphs. Early approaches rely on heuristic perturbations. CF-GNNExplainer and CF² [16, 31] learn a continuous mask over the existing edge set, which restricts the search space to edge deletions only and, therefore, cannot introduce the structural motifs that a valid counterfactual may require. C2Explainer [19] broadens this space to edge additions and node-feature perturbations, and XPlore [32] pursues the same direction, driving gradient-guided perturbations of the adjacency and the node-feature matrix jointly. InduCE [47] instead attacks the per-instance optimization itself, learning a reinforcement-learning policy over edge additions and deletions that transfers to unseen inputs without instance-specific training. It targets node rather than graph classification and is therefore outside the scope of our comparison. All of these methods operate on a binary adjacency matrix and thus cannot predict categorical edge types, i.e. they cannot distinguish a single, double, or aromatic bond between the same pair of atoms, a distinction that is essential for chemical validity. UCExplainer [33] is the exception among the mask-based methods. It perturbs node features, topology, and bond types alike and is therefore, together with GDCE-I, the only method that can be evaluated against an edge-feature-aware classifier.

A complementary line explains the model rather than the instance. Global graph counterfactual explanation [48] examines a compact set of subgraph substitution rules that flip the prediction across many graphs at once, trading per-instance faithfulness for coverage. We target instance-level counterfactuals throughout.

Generative methods for Graphs. A second line of work generates counterfactuals from a learned model of the data distribution. CLEAR [18] employs a graph variational autoencoder (VAE) that decodes a Bernoulli adjacency matrix together with node features but does not assign bond types. CGCF [14] also traverses a continuous VAE latent space and, unlike CLEAR, it decodes categorical edge types, but it shares the more fundamental drawback that a continuous latent relaxation is a questionable model for the highly constrained, discrete nature of graphs. RSGG-CE [49] instead adopts a GAN, again without categorical edge types. In the molecular domain, MEG [50] employs a reinforcement-learning agent that edits the molecular graph with valence-valid operations. A related line of work for molecules operates outside the graph representation altogether, namely MMACE, which enumerates chemically valid neighbors through SELFIES string mutations. LLM-GCE [51] also operates on textual input, generating counterfactuals with large language models for Molecules. Diffusion-based methods are the most directly related. D4Explainer [17] trains a discrete denoising diffusion model with a counterfactual loss term to generate in-distribution explanations. However, it operates on a binary adjacency matrix without categorical nodes or edge types, and its counterfactual objective is incorporated only at training time.

Evaluating Explanations. A major challenge in designing explanation techniques is the issue of evaluation. Ground-truth explanations are rarely available, and

the notion of a good explanation depends on the use case. The question of evaluation has been central, with early foundations including [52] and [53], with the latter proposing a set of practical desiderata for explanation techniques. While the evaluation of explanations has been extensively covered for attribution methods (e.g. [54, 55]), for visual counterfactuals in [42], and counterfactuals for tabular data (e.g. [37]), there have been comparatively fewer such studies in the context of graph counterfactual explanations.

In summary, while similar metrics to the one suggested by us have been proposed in the context of tabular and vision data, previous work on graph counterfactual evaluation has predominantly focused on a form of *flip-ratio* and *sparsity*.

3. Desiderata of Counterfactuals

To enhance the usefulness of graph counterfactual explainers (GCEs), and analogous in large parts to the desiderata-driven framework established for visual counterfactuals by Bender et al. [42], we take as a starting point the holistic explanation desiderata formulated by Swartout and Moore (1993) [53]. These desiderata, originally proposed in the context of explaining expert systems, are ‘fidelity’, ‘understandability’, ‘sufficiency’, ‘low construction overhead’, and ‘efficiency’. In the following, we contribute an instantiation of the first three desiderata specifically tailored to counterfactual explanations on graphs and operationalize them using concrete metrics: Non-Adversarial Rate (NA), a three-fold Sparsity breakdown (Edge, Node, Edge Type), and Non-Adversarial Flip Rate (NAFR).

3.1. Fidelity

For an explanation to be useful, it should faithfully describe what the model does. As noted in [53], an incorrect or misleading explanation is worse than no explanation at all. Instantiating this desideratum to counterfactual explanations following [42], we propose the following formalization: Assuming a factual graph G and a classifier f that varies locally following some dominant direction w , the counterfactual graph G' is faithful, i.e., represents what the model does, if it meets the following three criteria:

1. The counterfactual should be plausible and thus be on the data manifold, i.e., $G' \in \mathcal{M}$, adhering to structural and domain-specific rules (such as valency).
2. The counterfactual’s immediate neighborhood, i.e. $\{\xi \in \mathcal{M} : d(\xi, G') < \delta\}$, should also be dominantly counterfactual.

In practice, testing these properties directly is not always possible due to discrete, combinatorial graph spaces and limited knowledge of $\mathcal{M}$.

It is interesting to note that most state-of-the-art GCEs, such as CLEAR [18] and D4Explainer [17], ensure (i) by limiting the search for G' to the data manifold $\mathcal{M}$ through a generative model or not all [16, 19, 33]. However, they do not

address (ii) and (iii), i.e., whether the transformation $G \mapsto G'$ is associated with a robust model response that allows resolutely crossing the decision boundary. Selecting any on-manifold transformation that flips the classifier (e.g., a minimal one) risks producing an adversarial example (cf. [56, 57]), as the counterfactual search may leverage spurious local variations in the classification function f that are not representative of how f varies more globally. Spurious local variations are commonplace in GNN classifiers [58] and also occur along the data manifold $\mathcal{M}$ [57]. Most existing counterfactual methods lack a mechanism to address potential spurious variations on the manifold.

Non-Adversarial Rate (NA) To operationalize *fidelity*, we evaluate whether counterfactuals occupy robust semantic basins rather than fragile adversarial pockets. Analogous to [42], we validate the generated graphs against a re-trained surrogate model f_{sur} (distilled independently from f). This measures the transferability of the counterfactual flip, confirming it relies on robust structural features rather than weight-specific noise:

$$\text{Fidelity: NA} = \frac{N_{\text{flipped_true}}}{N_{\text{flipped}}} \quad (1)$$

where $N_{\text{flipped_true}}$ represents the count of samples that also successfully flip the prediction of f_{sur} . Higher NA values indicate greater semantic stability and fidelity to the underlying decision process.

Further in the molecular domain, we will report **SMILES** ($\uparrow$), the fraction of counterfactuals that RDKit [59] can parse and sanitize under valence and bond-compatibility rules. SMILES is defined only on the two molecular datasets, Mutagenicity and Benzene.

3.2. Understandability

Understandability refers to whether an explanation is understandable to its intended recipient, who is usually a human. Explanations should be presented at an appropriate level of abstraction and be concise enough to be quickly assimilated. In the context of graph counterfactual explanations, an intrinsic level of understandability is guaranteed by the fact that generated counterfactuals live in the same input domain as factuais and reside on the data manifold $\mathcal{M}$ (i.e., inspectable discrete attributed graphs). Following [42], we capture the concise nature required for understandability through structural *sparseness* across the transformation $G \mapsto G'$.

To evaluate understandability fairly in the graph domain across different baselines is complicated by an asymmetry in their action spaces. Many baselines operate on a binary adjacency matrix and can only add or delete edges, whereas GDCE-I additionally edits atom and bond types. Reducing both to a single distance is therefore misleading. Further predicting the existence of a bond is chemically ambiguous (a single and a double bond between the same atoms are indistinguishable in a binary adjacency matrix), so any proximity computed on that representation abstracts away precisely the information that determines molecular identity. We therefore decompose structural minimality into three

complementary sparsity metrics that jointly cover the attributed perturbation space while preserving comparability with edge-only baselines:

Edge Sparsity. Topological minimality measured as the fraction of added or removed edges relative to the original graph’s edge set:

$$\text{Edge Sparsity}(G, G') = \max(1 - \frac{|E \triangle E'|}{|E|}, 0) \quad (2)$$

where $\triangle$ denotes the symmetric difference of the edge sets. The term $\frac{|E \triangle E'|}{|E|}$ is sometimes reported under the name ‘Modification Ratio’ in related work papers, but we want to emphasize the connection between sparsity and understandability also elaborated in other interpretability research like [60].

Node Sparsity. The fraction of node-type substitutions across all nodes, calculated relative to the node feature matrix:

$$\text{Node Sparsity}(G, G') = 1 - \frac{\sum_{i=1}^{|X|} \mathbb{1}[x_i \neq x'_i]}{|X|} \quad (3)$$

Edge Type Sparsity. Semantic changes on structurally preserved edges, measuring bond-type reassignments on retained edges:

$$\text{Edge Type Sparsity}(G, G') = 1 - \frac{|\{e \in E \cap E' \mid \text{attr}(e) \neq \text{attr}'(e)\}|}{|E \cap E'|} \quad (4)$$

These three metrics isolate topological modifications, node-attribute substitutions, and edge-type shifts to provide a fine-grained quantification of concise, understandable edits. To illustrate the difference between fidelity (measured via NA) and understandability (measured via Sparsity), we provide practical examples in Table 1.

3.3. Sufficiency

Following [42], sufficiency requires charting the decision boundary globally across numerous distinct instances rather than exploiting localized or isolated successes. We operationalize sufficiency via the **Non-Adversarial Flip Rate (NAFR)**.

Non-Adversarial Flip Rate (NAFR) Bridging both sufficiency and fidelity [42], NAFR tracks the absolute proportion of generated samples across the dataset that simultaneously alter the target model’s prediction and induce a genuine, transferable semantic shift confirmed by the surrogate classifier f_{sur} :

$$\text{Sufficiency:NAFR} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}(f(G'_i) = y_t \wedge f_{sur}(G'_i) = y_t) \quad (5)$$

By combining coverage and transferability into a single metric, NAFR isolates functional, structurally coherent graph counterfactuals from those that either fail to flip the target model or merely exploit imperceptible adversarial vulnerabilities.

Faithful (High NA) but not Understandable (Low Sparsity)	<i>A counterfactual that robustly flips both the target model and the surrogate model (high NA), but achieves this via an extensive graph-wide rewrite involving widespread topological rewiring, node substitutions, and edge-type modifications (poor Edge, Node, and Edge Type Sparsity) that overwhelm human inspection.</i>
Understandable (High Sparsity) but not Faithful (Low NA)	<i>A counterfactual with minimal structural edits (e.g., changing a single atom type or edge, yielding high Sparsity) that successfully flips the target classifier f, but fails to flip the surrogate model f_{sur} (low NA) because it exploits a fragile, weight-specific adversarial vulnerability rather than a robust decision boundary.</i>
Faithful (High NA) but Insufficient (Low NAFR)	<i>A counterfactual method that produces stable, non-adversarial edits when it succeeds (high NA), but fails to generate valid counterfactuals for a large fraction of the dataset, resulting in a low absolute Non-Adversarial Flip Rate (NAFR) and failing to globally map the classifier’s behavior.</i>

Table 1: Practical examples of graph counterfactuals that meet only some of the operationalized desiderata (NA, Sparsity, NAFR), adapting the conceptual framework of [42].

4. Method

Discrete Denoising Diffusion on Graphs. Let a graph be defined as $G = (X, E)$, where $X \in \{0, 1\}^{n \times a}$ represents the categorical node features and $E \in \{0, 1\}^{n \times n \times b}$ denotes the categorical edge features for n nodes.

The fundamental objective of the forward diffusion process is to progressively transform an input graph $G \sim p_{data}$ into a sequence of increasingly noisy states, ultimately reaching a pure noise distribution G^T at the final timestep T . While standard continuous diffusion models corrupt data by injecting Gaussian noise, applying this paradigm directly to adjacency matrices destroys the inherent sparsity of graphs, resulting in dense, fractional structures. To address this, discrete denoising diffusion models, such as DiGress [15], define Markov transitions directly over categorical node and edge types via transition matrices $[Q_X^t]_{ij} = q(x^t = j | x^{t-1} = i)$ and $[Q_E^t]_{ij} = q(e^t = j | e^{t-1} = i)$. Similar to the continuous DDPM [61] forward process, the forward diffusion in discrete spaces is then defined as a Markovian process that iteratively adds noise at each timestep

$$q(G^t | G^{t-1}) = (X^{t-1} Q_X^t, E^{t-1} Q_E^t). \quad (6)$$

Due to the Markovian property, it is not necessary to apply noise recursively. By defining the cumulative transition matrix $\bar{Q}^t = Q^1 Q^2 \dots Q^t$, we can directly sample the noisy graph at any arbitrary timestep t from the initial clean graph G :

$$q(G^t | G) = (X \bar{Q}_X^t, E \bar{Q}_E^t). \quad (7)$$

The goal of the generative model is to reverse this forward process. To do so, DiGress formulates the reverse process as a product of independent transitions over all nodes and edges

$$p_\theta(G^{t-1}|G^t) = \prod_{1 \leq i \leq n} p_\theta(x_i^{t-1}|G^t) \prod_{1 \leq i, j \leq n} p_\theta(e_{ij}^{t-1}|G^t), \quad (8)$$

where the individual marginals are computed by marginalizing over the clean state predictions $\hat{p}_i^X(x)$ produced by the denoising neural network

$$p_\theta(x_i^{t-1} | G^t) = \sum_x q(x_i^{t-1} | x_i^t, x_i^0 = x) \cdot \hat{p}_i^X(x) \quad (9)$$

and $p_\theta(e_{ij}^{t-1} | G^t)$ is computed accordingly.

Classifier-Free guidance. In conditional graph generation, traditional classifier-based guidance [62] relies on an auxiliary property regressor evaluated on noisy intermediate graphs G^t [15]. However, assigning meaningful continuous properties to such invalid, highly corrupted states is fundamentally ill-posed and yields unreliable guidance gradients. Classifier-Free Guidance (CFG) [21, 22] circumvents this issue by jointly training a single generative model for both conditional and unconditional generation. During training, the conditioning variable y (e.g., a target class or continuous property) is randomly replaced by a null token $\emptyset$ with a predefined probability p_{uncond} . During inference, the model extrapolates toward the conditional objective governed by a guidance scale s . Formally, the conditioned reverse process $p_\theta(G^{t-1}|G^t, y)$ is factorized over all nodes and edges under the assumption of conditional independence

$$p_\theta(G^{t-1}|G^t, y) = \prod_{1 \leq i \leq n} p_\theta(x_i^{t-1}|G^t, y) \prod_{1 \leq i, j \leq n} p_\theta(e_{ij}^{t-1}|G^t, y). \quad (10)$$

To compute the individual marginals, the model predicts the clean state from the noisy intermediate graph. For a node i , this marginalization is given by

$$p_\theta(x_i^{t-1} | G^t, y) = \sum_x q(x_i^{t-1} | x_i^t, x_i^0 = x) \cdot f_\theta(x_i^0 = x | G^t, y) \quad (11)$$

where the clean state prediction $\hat{p}_\theta$ incorporates the classifier-free guidance mechanism. By linearly combining the conditional and unconditional predictions with the guidance scale s , the guided prediction is formulated as

$$f_\theta(x_i^0 = x | G^t, y) = p_\theta(x_i^0 | G^t) + s(p_\theta(x_i^0 | G^t, y) - p_\theta(x_i^0 | G^t)) \quad (12)$$

and the edge marginals $p_\theta(e_{ij}^{t-1} | G^t, y)$ are computed accordingly.

Categorical Sampling via the Gumbel-Max Trick. Both the forward and the reverse process require drawing samples from categorical distributions over node and edge types. Our inversion scheme relies on making this sampling step reparameterizable, so that its stochasticity can be recorded and later replayed. This is achieved with the Gumbel-Max trick. Let $p \in \Delta^{K-1}$ be a categorical distribution over K classes, and let $g = (g_1, \dots, g_K)$ be a vector of independent

standard Gumbel variables, $g_k \sim \text{Gumbel}(0, 1)$, which can be sampled as $g_k = -\log(-\log u_k)$ with $u_k \sim \mathcal{U}(0, 1)$. Then

$$\arg \max_k (\log p_k + g_k) \sim \text{Categorical}(p), \quad (13)$$

that is, perturbing the log-probabilities with Gumbel noise and taking the argmax yields an exact sample from p . Crucially, this decomposes a categorical draw into a deterministic component, the log-probabilities $\log p$ that depend on the conditioning, and a stochastic component, the noise g that is independent of the content. Holding g fixed while altering $\log p$ therefore changes the resulting sample only at positions where the shift in log-probabilities is large enough to move the argmax to a different class. This is the mechanism GDCE-I exploits to invert the discrete diffusion process in Section 4.1.

4.1. GDCE-I

Building upon the theoretical foundations of discrete diffusion and classifier-free guidance, we present Graph Diffusion Counterfactual Explanations via Edit-Friendly Inversion (GDCE-I). As a fundamental prerequisite to our framework, we first train a conditional discrete diffusion model on the target dataset. The conditioning signal utilized during this training phase can be derived directly from the predictions of the pre-trained black-box model, a process we term the distillation of the classifier into the discrete diffusion model. Notably, this conditional generative framework is highly versatile. The conditioning variables are not restricted to categorical classifier labels, but can seamlessly incorporate ground-truth variables, continuous physical properties (e.g., molecular logP), or even multi-dimensional property vectors. Once this conditional generative backbone is fully established, we can employ our inversion procedure to generate precise counterfactuals.

In continuous Denoising Diffusion Probabilistic Models (DDPMs), recent work has shown that the sequence of noise vectors drawn during the reverse process encodes the structural backbone of the generated sample, so that fixing this noise while altering the conditioning enables semantically meaningful edits that tightly preserve the original structure [63]. We translate this “edit-friendly” paradigm to the discrete graph domain. The central obstacle is that discrete diffusion samples from categorical transitions rather than adding Gaussian noise. We therefore realize an analogous noise space through the Gumbel-Max trick of Section 4, which separates each categorical draw into deterministic log-probabilities and content-independent Gumbel noise (Eq. 13).

A naive realization would sample fresh Gumbel noise along the forward chain and hope the reverse process retraces it. This does not hold in general because the noise would be recorded against the forward transitions Q^t , yet replayed against the learned reverse posterior $p_\theta(\cdot \mid G^t, y)$, which is a different distribution. We instead construct an edit-friendly posterior noise space that guarantees exact reconstruction, using the truncated-Gumbel (A^* -sampling) construction [23].

Reference trajectory. Given an input graph $G = (X, E)$ with its (distilled) condition y , we first draw a single reference trajectory $\hat{G}^0, \hat{G}^1, \dots, \hat{G}^T$ with $\hat{G}^0 = G$ by iterating the discrete forward process of Eq. 6, retaining only the visited states $\hat{G}^t = (\hat{X}^t, \hat{E}^t)$.

Posterior noise recording. We then traverse the trajectory in the reverse direction and, for every node and edge independently, record the Gumbel noise that forces the guided reverse posterior to reproduce the reference state. We describe the construction for a single categorical variable (one node; edges are analogous). At the reverse step from $t = s+1$ to s , let $\ell \in \mathbb{R}^K$ denote that node’s guided reverse-posterior log-probabilities over the K classes, $\ell = \log p_\theta(X^s \mid \hat{G}^{s+1}, y)$, and let v^* be the class realized at that node in the reference state $\hat{X}^s$. We seek Gumbel noise $g \in \mathbb{R}^K$ that selects this class under the Gumbel-Max rule of Eq. 13,

$$\arg \max_k (\ell_k + g_k) = v^*. \quad (14)$$

Following the truncated-Gumbel (A*-sampling) construction [23], we build the perturbed log-probabilities $\phi_k = \ell_k + g_k$ directly. Let $Z = \log \sum_k e^{\ell_k}$ be the log-normalizer (here $Z = 0$, since p_θ is normalized) and let $M = Z + \gamma_0$ with $\gamma_0 \sim \text{Gumbel}(0, 1)$ be the maximal perturbed value. We assign M to the target class and draw every other class as an independent $\text{Gumbel}(\ell_k)$ value truncated to lie below M :

$$\phi_k = \begin{cases} M, & k = v^*, \\ -\log(e^{-(\ell_k + \gamma_k)} + e^{-M}), & k \neq v^*, \end{cases} \quad \gamma_k \sim \text{Gumbel}(0, 1). \quad (15)$$

By construction $\arg \max_k \phi_k = v^*$. Since $\phi = \ell + g$, the reusable, condition-independent noise is recovered by subtracting the log-probabilities,

$$g = \phi - \ell. \quad (16)$$

Collecting g over all nodes (resp. edges) yields the node noise g_s^X (resp. the edge noise g_s^E , symmetrized across the adjacency). We store all recorded noise in a library $\mathcal{G} = \{(g_s^X, g_s^E)\}_{s=0}^{T-1}$ together with the reference states.

Guided generation.. To produce a counterfactual, we replay the reverse process with the same recorded noise but a new target condition y' . Stepping from t to $t-1$, we compute the guided posterior $p_\theta(X^{t-1} \mid G^t, y')$ and sample deterministically with the stored noise,

$$X^{t-1} = \text{one_hot}\left(\arg \max (\log p_\theta(X^{t-1} \mid G^t, y') + g_{t-1}^X)\right), \quad (17)$$

and reconstruct E^{t-1} symmetrically from g_{t-1}^E . By construction, replaying with the original condition ($y' = y$) recovers the input graph exactly. Under a changed condition y' , a position changes only where the guidance-induced shift in log-probabilities is large enough to move the argmax past the recorded margin. Edits are therefore concentrated precisely where the target property

demands them, yielding sparse, in-distribution counterfactuals, and resolving the mismatch of the naïve scheme, whose noise is recorded and replayed against different distributions. We show a comparison in 5.3.

Dynamic budget search.. Replaying the full trajectory from T can introduce more changes than are needed to flip the classifier. We therefore introduce a budget $\tau \in \{1, \dots, T\}$: rather than replaying from T , we start the guided reverse pass from the reference state $\hat{G}^\tau$ and denoise down to G^0 , reusing $\{(g_s^X, g_s^E)\}_{s < \tau}$. The framework searches over increasing τ and returns the counterfactual at the smallest budget for which $f(G^0) = y'$, which identifies the minimal structural deviation required to alter the prediction. Unlike DiGress, which draws plain categorical samples during generation [15], GDCE-I’s Gumbel-Max reparameterization is what renders the discrete trajectory invertible in the first place. The complete procedure, namely the reference trajectory, posterior recording, and budgeted guided replay, is summarized in Algorithm 1 (Appendix Appendix A).

5. Experiments

To comprehensively evaluate the efficacy of our proposed framework, we design our experiments to answer the following two primary research questions:

- **Quantitative Benchmarking:** How does GDCE-I compare against state-of-the-art baseline explainers in terms of our defined evaluation framework on standard graph classification tasks?
- **Qualitative Analysis:** Can GDCE-I produce understandable edits?

5.1. Experimental Setup

Datasets. For the discrete graph classification tasks, we evaluate our method on two widely adopted molecular benchmarks: Mutagenicity [24] and Benzene [25, 26]. In these datasets, graphs represent chemical compounds. The binary label of Mutagenicity indicates a mutagenic effect, whereas Benzene is a structural attribution benchmark whose label indicates the presence of a benzene ring.

The focus on molecules is deliberate, and it concerns what can be measured rather than what the method supports. GDCE-I operates on categorical node and edge types and assumes nothing chemistry-specific. Molecular graphs, however, come with distributional constraints like valence rules. In-distribution behavior is therefore observable.

We additionally evaluate on two non-molecular benchmarks from the TU-Dataset collection [64], both without edge types. PROTEINS [27, 28] represent proteins as graphs of secondary-structure elements, with three categorical node labels (helix, sheet, turn) and a binary label separating enzymes from non-enzymes. TWITTER [30, 29] represents a tweet as a word co-occurrence graph, where nodes are words, and the binary label is the sentiment of the tweet.

Baselines. We benchmark GDCE-I against graph counterfactual explainers that produce instance-level counterfactuals for graph classification, spanning the heuristic/mask-based (CF^2 [31], C2Explainer [19], XPlore [32], UCExplainer [33]) and generative (D4Explainer [17]) families of Section 2. All baselines are re-evaluated under a single common protocol: identical dataset preprocessing, test split, sampling seeds, and metric definitions. Crucially, every method is optimized against and evaluated by the same classifier. We train a binary-adjacency GCN, which every method can consume and against which all of them are measured, and additionally an edge-feature-aware GINE, reserved for the two methods that predict bond types (GDCE-I and UCExplainer). Both are trained from scratch on the shared training split. Architectures and accuracies are given in appendix Appendix B. PROTEINS and TWITTER carry no edge types and, therefore, have a GCN classifier only.

Counterfactuals evaluation. A counterfactual explanation must name a realizable modification of the input, so we require the counterfactual to lie in the same space as the input: one-hot atom types, one-hot bond types, and binary edges. Methods that optimize a continuous relaxation are therefore discretized before any metric is computed. Node and bond vectors are projected back onto the simplex by arg max, soft edge masks are thresholded at 0.5, and the Flip Rate is re-evaluated on the resulting graph. Two baselines are affected: UCExplainer returns node and bond vectors with several simultaneously active entries, and D4Explainer evaluates its flip on sigmoid-weighted edges rather than on a thresholded adjacency. For fair comparison, we report the post-discretisation value. Further, CF^2 , C2Explainer, XPlore and D4Explainer operate on a binary adjacency and predict no bond types, so SMILES reconstruction requires a convention. CF^2 only deletes edges: every surviving edge retains the bond order of the input, and no assignment is needed. C2Explainer, XPlore and D4Explainer can additionally add edges, which carry no bond label. We assign these a single bond. This is the most frequent bond type in both datasets (81.3% of bonds in Mutagenicity, 56.3% in Benzene) and, being the lowest-valence option, the assignment is least likely to trigger a valence violation.

5.2. Quantitative Evaluation on Graph Classification

Table 2 reports every method against the same classifier per dataset, so no difference between rows can be attributed to a differing classifier. For every method, we evaluate five runs of 100 molecules drawn from the test set under identical seeds. All entries are means $\pm$ standard deviations over those runs. The only exception is the PROTEINS dataset, which is smaller, and we therefore evaluate on the whole test set.

GDCE-I attains the highest Flip Rate on all four datasets. This is unchanged under NAFR, which additionally requires the flip to be confirmed by the control judge, and GDCE-I likewise leads the SMILES column on both datasets on which it is defined. It therefore makes the strongest case for altering the classifier’s prediction while returning counterfactuals that remain on the data

manifold, and it does so while operating, together with UCExplainer, in the largest edit space of the compared methods.

UCExplainer attains the maximum edge score of 1.000 but just because it fools the classifier by adding entry in one-hot encoded vectors which the classifier never saw. Among the methods that do edit topology, GDCE-I preserves the most of it on Mutagenicity, Benzene, and TWITTER. On PROTEINS, XPlore is ahead (0.832 vs 0.747) but just with a lower NAFR 0.649 against GDCE-I 0.747. On atom types, GDCE-I leads outright on three of the four datasets.

Method	desiderata						
	sufficiency		understandability			fidelity	
	FR (†)	NAFR (†)	EdgeSp. (†)	NodeSp. (†)	EdgeTypeSp. (†)	NA (†)	SMILES (†)
Mutagenicity							
CF ² [31]	0.622±0.029	0.510±0.019	0.396±0.020	—	—	0.820	0.508±0.016
C2Explainer [19]	0.688±0.016	0.558±0.028	0.000 [†]	—	—	0.811	0.002±0.004
XPlore [32]	0.922±0.018	0.586±0.069	0.613±0.056	—	—	0.635	0.050±0.017
UCExplainer (GCN) [33]	0.158±0.044	0.148±0.043	1.000 ±0.000	0.754±0.036	—	0.937	0.088±0.032
UCExplainer (GINE)	0.260±0.044	0.186±0.035	1.000 ±0.000	0.738±0.050	0.910±0.033	0.715	0.068±0.013
D4Explainer [17]	0.138±0.029	0.132±0.033	0.597±0.101	—	—	0.957	0.120±0.040
GDCE-I (Ours, GCN)	0.970 ±0.016	0.886 ±0.029	0.657±0.011	0.892±0.009	0.993±0.000	0.913	0.744 ±0.017
GDCE-I (Ours, GINE)	0.770±0.049	0.628±0.068	0.680±0.016	0.919 ±0.006	0.994 ±0.002	0.816	0.534±0.069
Benzene							
CF ²	0.566±0.021	0.550±0.007	0.391±0.005	—	—	0.972	0.000±0.000
C2Explainer	0.666±0.038	0.610±0.032	0.910±0.010	—	—	0.916	0.092±0.022
XPlore	0.670±0.021	0.598±0.004	0.780±0.038	—	—	0.893	0.206±0.036
UCExplainer (GCN)	0.202±0.023	0.184±0.029	1.000 ±0.000	0.818±0.027	—	0.911	0.092±0.022
UCExplainer (GINE)	0.558±0.029	0.508±0.036	1.000 ±0.000	0.459±0.020	0.369±0.014	0.910	0.000±0.000
D4Explainer	0.290±0.054	0.290±0.054	0.597±0.080	—	—	1.000	0.290±0.054
GDCE-I (Ours, GCN)	0.888±0.037	0.858±0.046	0.915±0.011	0.910 ±0.004	0.998 ±0.001	0.966	0.360 ±0.044
GDCE-I (Ours, GINE)	0.948 ±0.020	0.938 ±0.026	0.947±0.007	0.882±0.005	0.989±0.003	0.989	0.286±0.039
PROTEINS							
CF ²	0.759	0.730	0.523	—	—	0.962	—
C2Explainer	0.431	0.374	0.365	—	—	0.868	—
XPlore	0.805	0.649	0.875	—	—	0.807	—
UCExplainer (GCN)	0.241	0.224	1.000	0.773	—	0.929	—
D4Explainer	0.132	0.121	0.832	—	—	0.917	—
GDCE-I (Ours, GCN)	0.851	0.747	0.762	0.872	—	0.878	—
TWITTER							
CF ²	0.194±0.019	0.130±0.027	0.213±0.037	—	—	0.670	—
C2Explainer	0.054±0.021	0.036±0.018	0.313±0.133	—	—	0.667	—
XPlore	0.142±0.029	0.082±0.016	0.621±0.049	—	—	0.582	—
UCExplainer (GCN)	0.532±0.069	0.474±0.061	1.000 ±0.000	0.522 ±0.066	—	0.891	—
D4Explainer	0.000±0.000	0.000±0.000	—	—	—	—	—
GDCE-I (Ours, GCN)	0.996 ±0.005	0.954 ±0.015	0.957±0.009	0.496±0.032	—	0.958	—

Table 2: All metrics are oriented so that higher is better (†); subscripts give the standard deviation over five runs of 100 graphs. The three sparsity columns are 1 minus the corresponding modification rate, so 1 means that part of the graph was left untouched. SMILES is defined on the molecular datasets only. [†]clamped at 0; the raw modification rate is 1.390. PROTEINS is a single run over the complete test split and admits no spread.

GDCE-I also allocates its edit budget to the property that defines each dataset. On Mutagenicity, it spends the budget on topology, preserving 0.892 of the atom types but only 0.657 of the edges, consistent with rearranging the substructures that carry mutagenicity. On Benzene, where the label is the presence of a benzene ring, it instead exploits an edit that no edge-only baseline can express and substitutes a ring carbon by another atom type. The ordering

reverses, with 0.947 of the edges but only 0.882 of the atom types left intact.

GDCE-I does not lead the NA column on the molecular datasets but stays competitive. One should note that NA is conditioned on the flips a method produces. D4Explainer’s 1.000 on Benzene is computed over the 29.0% of graphs it flips at all, whereas GDCE-I’s 0.966 is computed over 88.8%. Read together with NAFR, which shares its denominator with FR, the two columns separate how often a method flips from how often its flips survive a change of judge.

The target classifier is not a neutral choice. Because the shared protocol is what makes Table 2 comparable, it is worth stating how much it changes. Re-running each baseline against *its own* paper oracle on the same splits, seeds, and metric definitions, exchanging nothing but the classifier, moves the Flip Rate by up to 0.30 on Mutagenicity: C2Explainer falls from 0.984 to 0.688, D4Explainer from 0.296 to 0.154 and CF^2 from 0.820 to 0.622. The effect is not uniform in sign, as UCEExplainer gains under the shared judge. The largest losses fall on precisely those methods that report a near-perfect Flip Rate against their own oracle, whereas GDCE-I moves by at most 0.06 in either direction ($0.908 \rightarrow 0.970$ on Mutagenicity).

Sparsity behaves differently. For most methods, structural cost is close to classifier-invariant: CF^2 ($0.600 \rightarrow 0.604$), D4Explainer ($0.429 \rightarrow 0.384$) and GDCE-I ($0.344 \rightarrow 0.343$) report essentially the same Edge Sparsity on Mutagenicity under either classifier. C2Explainer is the exception, rising from 0.084 to 1.390, reproduced as $15.3\times$ in a paired control that exchanges nothing but the classifier.

5.3. Ablation Studies

The role of the inversion. We compare three different sampling schemes under the same training run, same checkpoint, same shared GCN classifier, guidance scale $s = 3$, same dynamic budget search, and 5×100 Mutagenicity samples. Firstly **no inversion**, which perturbs the source to G_τ along the forward trajectory and then samples the reverse process freshly. Secondly **naive inversion**, which records and re-injects a fresh Gumbel sample rather than the posterior-consistent one, and **posterior inversion** (GDCE-I), which re-injects the exact edit-friendly noise 4.1. Table 3 reports the outcome under the same metrics.

Variant	desiderata							$\bar{\tau}$
	sufficiency		understandability			fidelity		
	FR ($\uparrow$)	NAFR ($\uparrow$)	EdgeSp. ($\uparrow$)	NodeSp. ($\uparrow$)	EdgeTypeSp. ($\uparrow$)	NA ($\uparrow$)	SMILES ($\uparrow$)	
No inversion	1.000 ± 0.000	0.876 ± 0.027	0.675 ± 0.028	0.923 ± 0.008	0.997 ± 0.001	0.876 ± 0.027	0.804 ± 0.025	11
Naive inversion	0.996 ± 0.005	0.858 ± 0.019	0.592 ± 0.033	0.907 ± 0.007	0.995 ± 0.002	0.861 ± 0.018	0.832 ± 0.036	36
Posterior (GDCE-I)	0.970 ± 0.016	0.886 ± 0.029	0.657 ± 0.011	0.892 ± 0.009	0.993 ± 0.000	0.913 ± 0.029	0.816 ± 0.021	51

Table 3: Inversion ablation on Mutagenicity against the shared GCN classifier (5×100 , $s = 3$), reported under the metrics of Table 2; $\bar{\tau}$ is the median flip budget. All three variants come from the same training run and checkpoint and differ only in the noise treatment, and the posterior row is the GDCE-I (GCN) row of Table 2. The column-wise best is in **bold**.

All three are strong counterfactual generators, and no desideratum separates them. They flip almost every instance (1.000, 0.996, 0.970), and once the flip

has to survive the control judge, they stay within one standard deviation of one another (0.876, 0.858, 0.886). The joint criterion of being both a flip and a molecule falls within 0.03 (0.804, 0.832, 0.816). On the understandability axes, the spread reaches at most two standard deviations: no inversion preserves the most topology (0.675 against 0.657 and the most atom types (0.923 against 0.892 for posterior inversion).

What the columns separate is how the flip is obtained, and the flip budget makes it visible. Without noise recording, the reverse process is freshly resampled, diverges almost immediately from the source, and finds a flip at a median budget of $\tilde{\tau} = 11$. Posterior inversion reconstructs the reference exactly at small τ and therefore cannot flip until the budget grows to $\tilde{\tau} = 51$. Naive inversion, which re-injects noise that the reverse posterior was never conditioned on, sits between the two at 36. The choice is thus a trade-off rather than a ranking. Posterior inversion is the only scheme that reconstructs, whereas no inversion reaches a flip at roughly a fifth of the budget.

Budget, molecule size, and edit size. The budget search assumes that a smaller τ yields a smaller edit. Testing that assumption, we observe (Figure 3) that this holds for all variants and also that the initial graph size sets the tone of how much budget is needed to find the counterfactual.

Posterior inversion lies below both alternatives at every budget. This is what exact reconstruction buys at the same budget on the same molecule. The guided replay departs from the source itself rather than from somewhere near it.

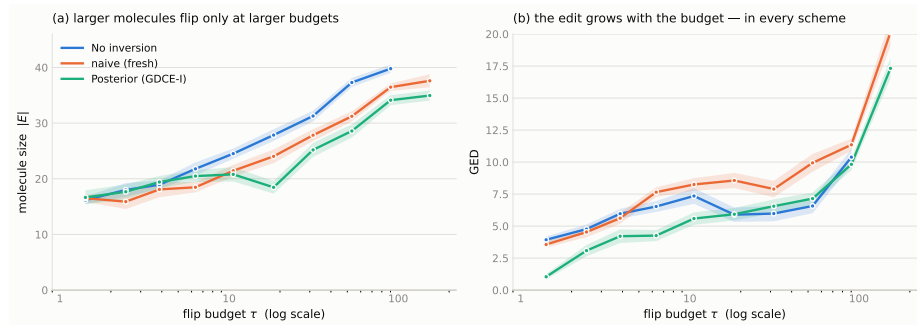


Figure 3: Flip budget τ on the horizontal axis of both panels (Mutagenicity, shared GCN classifier, $s = 3$, successful counterfactuals; binned means ± 1 SEM, log scale). (a) Moleculesize against τ . (b) Absolute edit size against τ .

Guidance strength.. The guidance scale s governs how forcefully generation is steered toward the target class, and sweeping it over $s \in \{1, 3, 5\}$ separates two effects that move in opposite directions. Raising s improves every axis that measures success and economy at once. The Flip Rate (0.740 $\rightarrow$ 0.970 $\rightarrow$ 0.998), all three understandability axes (0.606 $\rightarrow$ 0.657 $\rightarrow$ 0.726 on edges, 0.882 $\rightarrow$ 0.892 $\rightarrow$ 0.905 on atom types, 0.991 $\rightarrow$ 0.993 $\rightarrow$ 0.995 on bond types) and the flip budget ($\tilde{\tau}$ 86 $\rightarrow$ 51 $\rightarrow$ 25). Stronger guidance reaches the target class earlier along the replay and, therefore, departs less far from the source.

Fidelity moves the other way. The rate at which a flip is transmitted to the control judge decreases with s ($0.944 \rightarrow 0.913 \rightarrow 0.879$), and so does the share of counterfactuals that RDKit can sanitize, conditioned on a flip ($0.852 \rightarrow 0.841 \rightarrow 0.776$). Both point at the same mechanism: forcing the classifier signal harder buys a high flip rate and sparsity, but pushes the sample off the data manifold.

A counterfactual is only useful if it fulfills all the requirements at once. The two columns that combine these requirements the best are in the middle. The joint flip-and-molecule rate at $s = 3$ (0.816 against 0.630 and 0.774) and the non-adversarial flip rate likewise (0.886 against 0.699 and 0.879). We adopt $s = 3$ for the main results.

	desiderata							
s	sufficiency		understandability			fidelity		$\bar{\tau}$
	FR ($\uparrow$)	NAFR ($\uparrow$)	EdgeSp. ($\uparrow$)	NodeSp. ($\uparrow$)	EdgeTypeSp. ($\uparrow$)	NA ($\uparrow$)	SMILES ($\uparrow$)	
1	0.740 $\pm$ 0.037	0.699 $\pm$ 0.029	0.606 $\pm$ 0.027	0.882 $\pm$ 0.009	0.991 $\pm$ 0.003	0.944$\pm$0.054	0.630 $\pm$ 0.042	86
3	0.970 $\pm$ 0.016	0.886$\pm$0.029	0.657 $\pm$ 0.011	0.892 $\pm$ 0.009	0.993 $\pm$ 0.000	0.913 $\pm$ 0.029	0.816$\pm$0.021	51
5	0.998$\pm$0.004	0.894$\pm$0.023	0.726$\pm$0.028	0.905$\pm$0.005	0.995$\pm$0.003	0.896 $\pm$ 0.020	0.774 $\pm$ 0.025	24

Table 4: Guidance-scale sweep for GDCE-I (posterior inversion) on Mutagenicity against the shared GCN classifier (5×100), reported under the metrics of Table 2; $\bar{\tau}$ is the median flip budget. Column-wise best in **bold**.

5.4. Qualitative Analysis on Mutagenicity

To evaluate whether the generated counterfactuals provide meaningful insights, we conducted a qualitative analysis on the Mutagenicity dataset. This reference comprises aromatic and heteroaromatic chemical compounds classified according to their mutagenic effect on the Gram-negative bacterium *Salmonella typhimurium* [24]. Toxicophore derivation studies have identified multiple functional subgroups that are highly discriminatory for the classification of mutagenicity [24]. Among these, the aromatic nitro group, particularly the benzene- NO_2 motif, emerges as the most frequently occurring and highly predictive structural characteristic. Kazius et al. [24] report that 87% of the molecules containing this substructure are mutagens, a finding mirrored by our shared GCN classifier, which predicts 94.6% of the 669 molecules that carry this motif as mutagenic across the combined train, validation, and test sets.

The qualitative evaluation of counterfactual methods on this motif was first established by the authors of CF2 [31], who constructed a filtered subset called ‘‘Mutag0’’ [31]. Their analysis showed that purely deletion-based methods tend to simply remove a few critical edges to break the toxicophore, which fails to yield an explanation [31] by ending up in a fragmented counterfactual. However, despite identifying this flaw, CF2 itself suffers from similar limitations. As a mask-based attribution method, CF2 is designed primarily to identify and extract an existing factual subgraph rather than generate a counterfactual instance. Because its counterfactual reasoning strictly defines the counterfactual as the original graph minus the identified factual mask, CF2 is restricted to edge and node deletions. Consequently, while the method successfully highlights the

relevant toxicophore, it will not necessarily generate a counterfactual, which is an instance of the data manifold.

To evaluate whether GDCE-I is capable of solving these issues, we design a targeted experiment. We filter the test set to isolate molecules that contain both the benzene-NO₂ motif and are explicitly classified as mutagenic by the GNN classifier. Then we apply our framework to generate counterfactual explanations for this subset. A faithful counterfactual explainer should naturally recognize the causal significance of this motif and specifically alter the aromatic nitro group to flip the prediction from mutagenic to non-mutagenic.

Our empirical evaluation strongly validates this expectation. Of the 116 molecules in this targeted subset, GDCE-I alters the prediction of the classifier for 115 (99.1%). A detailed structural analysis further reveals that in 93.0% of the generated counterfactuals, the explainer explicitly modifies the benzene-NO₂ motif. 72.2% of the counterfactuals in this subset yield a valid SMILE. Figure 4 presents two representative examples. In the first, the method detoxifies the molecule by detaching the NO₂ group from the benzene ring and relocating it to a non-aromatic structural position. In the second, it performs a direct substitution of functional groups, replacing the mutagenic NO₂ group with a benign SO₂ group. Both strategies neutralize the primary toxicophore while strictly preserving the topological backbone of the original molecule, underscoring the capability of GDCE-I to deliver understandable counterfactuals.

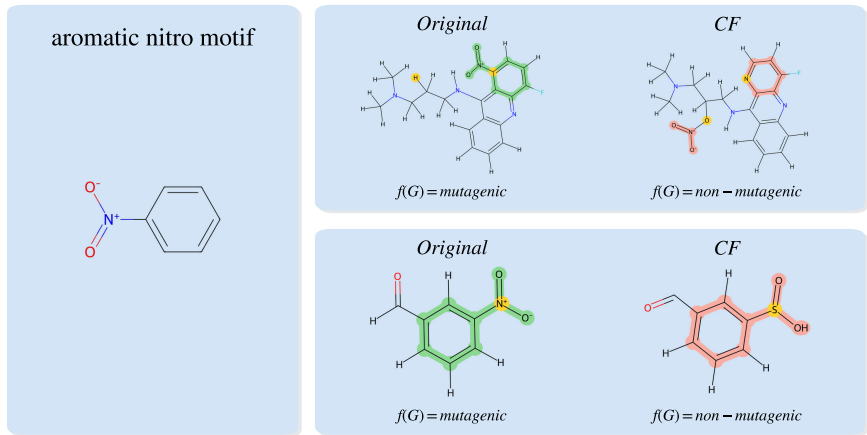


Figure 4: Qualitative counterfactuals on Benzene (GDCE-I (GCN), $s = 3$). Highlighting: *green* marks the original motif, *red* the remainings, and *yellow* the edits. (*Left*) The aromatic nitro motif. (*Top right*): The original mutagenic compound carries an NO₂ group connected to an aromatic ring. The generated counterfactual detoxifies the molecule by relocating the nitro group from the aromatic to a non-aromatic structural position. (*Bottom right*): GDCE-I performs a direct functional group substitution, replacing the mutagenic NO₂ group with a, non-mutagenic SO₂ group.

5.5. Qualitative Analysis on Benzene

We conduct another test on Benzene. A graph is exactly positive when it contains a benzene ring [25, 26]. Thus, a faithful explanation would destroy the benzene ring when the target class is negative and construct one when it is positive.

GDCE-I flips 94.8% of the sampled molecules (Table 2). Among the counterfactuals that produce a valid SMILES, 86.4% satisfy the ground-truth motif condition, i.e., the benzene ring is actually removed or created. In the positive-to-negative direction, this holds for every counterfactual. Crucially, 74.9% of all flips are obtained without inserting or deleting a single edge. GDCE-I breaks or completes the ring by substituting a ring atom, an edit that leaves the surrounding topology untouched.

Figure 5 shows one representative example per direction, both single-atom substitutions with Edge Sparsity and Edge Type Sparsity of exactly 1. In the Top right, carbon is replaced by nitrogen. Notably, the thiophene at the other end of the molecule, which is aromatic but not a benzene ring, is left untouched, so the edit targets the labeled motif rather than aromaticity in general. The second example runs the other way: a pyridine nitrogen is replaced by carbon, completing a benzene ring and flipping the prediction to the positive class. Together with the Mutagenicity analysis, this shows that GDCE-I localizes the class-defining substructure, and edits it data-manifold aware.

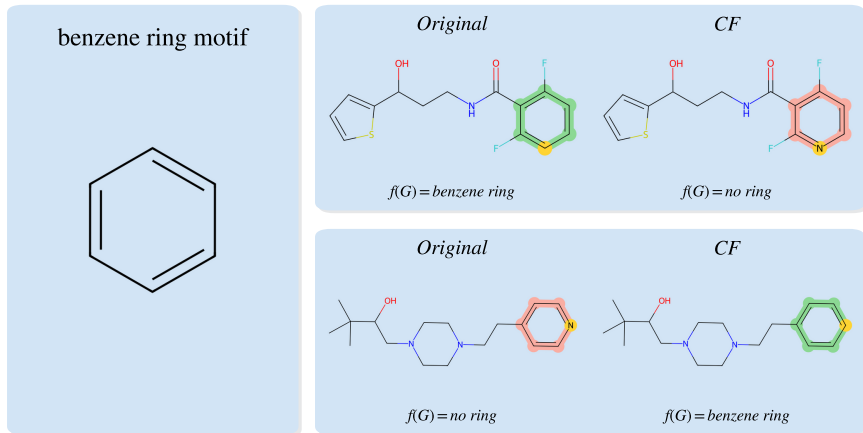


Figure 5: Qualitative counterfactuals on Benzene (GDCE-I (GINE), $s = 3$). Highlighting: *green* marks the original motif, *red* the remainings, and *yellow* the edits. (Left) The benzene-ring motif. (Top right): A ring carbon of the original is replaced by nitrogen, flipping the prediction to no benzene ring. (Bottom right): The reverse direction, where a nitrogen is replaced by carbon so that a benzene ring is completed.

6. Limitations

Our results should not be read as evidence that generative counterfactual explainers dominate heuristic ones. What a distribution-aware generator buys is that its edits are drawn from a model which learned the data distribution. Molecular graphs constrain it sharply, but on benchmarks whose distribution is not that unconstrained, such as synthetic graphs built from planted motifs, we expect modeling the data manifold buys nothing while its cost remains, and we would not expect GDCE-I to be the appropriate tool. Further, GDCE-I and other generative approaches require a conditional diffusion model trained per dataset, a cost the mask-based baselines do not incur at all. This rules the method out where an explanation is needed on a dataset for which no generative model exists, or it’s too hard to be trained, a regime in which mask-based explainers remain structurally advantaged regardless of the quality of their edits.

The counterfactuals we generate are edits to a molecular graph. Atom types, bond types, and topology. Many of the questions that motivate counterfactual reasoning in chemistry, from binding to pharmacokinetics, are not decided at that level, but by the three-dimensional conformations a compound adopts. A topological edit that flips a classifier trained on graphs therefore states a hypothesis about the molecule, not about its behavior in a conformation space.

7. Conclusion

We presented GDCE-I, a framework for graph counterfactual explanation built on discrete diffusion inversion. Its central component is a Gumbel-Max inversion that records the sampling noise of a reference trajectory in closed form, so that replaying the reverse process under the original condition reconstructs the input exactly, and replaying it under a changed condition alters only those positions where guidance is strong enough to move the argmax past the recorded margin. The budget parameter τ turns minimality into a search over how much of the trajectory is regenerated rather than a penalty term fixed at training time.

This addresses the two problems we identified. Because every edit is a step of a discrete diffusion model of the data, the counterfactual is held on the data manifold rather than merely close to the input. Secondly, because that model is defined over node types, bond types, and topology within a single generative process, the full edit space can be leveraged to find an explanation. The results follow from both. GDCE-I attains the highest Flip Rate of all compared methods on every benchmark (0.970 on Mutagenicity, 0.948 on Benzene, 0.851 on PROTEINS, 0.996 on TWITTER) and the highest non-adversarial flip rate throughout (0.886, 0.938, 0.747, 0.954). On the two molecular benchmarks, where membership of the data manifold can actually be tested, it returns the largest share of counterfactuals that are at once non-adversarial and a valid molecule (0.744 and 0.360).

Our second contribution is the evaluation itself. The metrics reported in this area are incomplete and inconsistent: the same names denote different amounts in papers, and different evaluation schemes are applied. We therefore derived desiderata for counterfactual explanation and defined suitable metrics for the graph domain.

8. Future Work

One possible direction of future work might be to also introduce diversity in the counterfactual search as done for tabular data, e.g., in [37] or for visual classifiers in [42]. This might enable us to systematically find secondary Clever Hans features [65] and remove them, e.g., with CFKD [66, 67] by adapting it plug-and-play from vision and tabular counterfactuals to graph counterfactuals. Another direction to explore would be applying the Gumbel softmax trick for editing with discrete diffusion to other inherently discrete domains like natural language or protein sequences.

Acknowledgments

We thank Klaus-Robert Müller for general discussion and guidance. We also thank Stefan Gugler for help and guidance for the qualitative experiments on Mutagenicity and Benzen. This work was partly funded by the German Ministry for Education and Research (under refs 01IS14013A-E, 01GQ1115, 01GQ0850, 01IS18056A, 01IS18025A, 13GW0744D, and BIFOLD25B) and by DFG. Sidney Bender was partially funded by Bosch-Siemens Haushaltsgeräte.

References

- [1] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, G. Monfardini, The graph neural network model, *IEEE Transactions on Neural Networks* 20 (1) (2009) 61–80. doi:10.1109/TNN.2008.2005605.
- [2] K. T. Schütt, H. E. Sauceda, P.-J. Kindermans, A. Tkatchenko, K.-R. Müller, Schnet – a deep learning architecture for molecules and materials, *The Journal of Chemical Physics* 148 (24) (2018) 241722. arXiv:https://pubs.aip.org/aip/jcp/article-pdf/doi/10.1063/1.5019779/16655678/241722_1_online.pdf, doi:10.1063/1.5019779. URL <https://doi.org/10.1063/1.5019779>
- [3] J. M. Stokes, K. Yang, K. Swanson, W. Jin, A. Cubillos-Ruiz, N. M. Donghia, C. R. MacNair, S. French, L. A. Carfrae, Z. Bloom-Ackermann, V. M. Tran, A. Chiappino-Pepe, A. H. Badran, I. W. Andrews, E. J. Chory, G. M. Church, E. D. Brown, T. S. Jaakkola, R. Barzilay, J. J. Collins, A deep learning approach to antibiotic discovery, *Cell* 180 (4) (2020) 688–702.e13. doi:10.1016/j.cell.2020.01.021.

- [4] Z. Wu, B. Ramsundar, E. Feinberg, J. Gomes, C. Geniesse, A. S. Pappu, K. Leswing, V. Pande, Moleculenet: a benchmark for molecular machine learning, *Chemical Science* 9 (2) (2018) 513–530. [arXiv:https://pubs.rsc.org/sc/article-pdf/9/2/513/5768099/c7sc02664a.pdf](https://pubs.rsc.org/sc/article-pdf/9/2/513/5768099/c7sc02664a.pdf), doi:10.1039/c7sc02664a.
URL <https://doi.org/10.1039/c7sc02664a>
- [5] H. Yuan, H. Yu, S. Gui, S. Ji, Explainability in graph neural networks: A taxonomic survey, *IEEE Trans. Pattern Anal. Mach. Intell.* 45 (5) (2023) 5782–5799. doi:10.1109/TPAMI.2022.3204236.
URL <https://doi.org/10.1109/TPAMI.2022.3204236>
- [6] E. Dai, T. Zhao, H. Zhu, J. Xu, Z. Guo, H. Liu, J. Tang, S. Wang, A Comprehensive Survey on Trustworthy Graph Neural Networks: Privacy, Robustness, Fairness, and Explainability, *Machine Intelligence Research* 21 (6) (2024) 1011–1061. doi:10.1007/s11633-024-1510-8.
URL <https://doi.org/10.1007/s11633-024-1510-8>
- [7] P. E. Pope, S. Kolouri, M. Rostami, C. E. Martin, H. Hoffmann, Explainability methods for graph convolutional neural networks, in: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 10764–10773. doi:10.1109/CVPR.2019.01103.
- [8] F. Wong, E. J. Zheng, J. A. Valeri, N. M. Donghia, M. N. Anahtar, S. Omori, A. Li, A. Cubillos-Ruiz, A. Krishnan, W. Jin, A. L. Manson, J. Friedrichs, R. Helbig, B. Hajian, D. K. Fiejtek, F. F. Wagner, H. H. Soutter, A. M. Earl, J. M. Stokes, L. D. Renner, J. J. Collins, Discovery of a structural class of antibiotics with explainable deep learning, *Nature* 626 (7997) (2024) 177–185. doi:10.1038/s41586-023-06887-8.
URL <https://doi.org/10.1038/s41586-023-06887-8>
- [9] S. Bach, A. Binder, G. Montavon, F. Klauschen, K.-R. Müller, W. Samek, On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation, *PloS one* 10 (7) (2015) e0130140.
- [10] T. Schnake, O. Eberle, J. Lederer, S. Nakajima, K. T. Schütt, K.-R. Müller, G. Montavon, Higher-order explanations of graph neural networks via relevant walks, *IEEE transactions on pattern analysis and machine intelligence* 44 (11) (2021) 7581–7596.
- [11] S. Wachter, B. Mittelstadt, C. Russell, Counterfactual explanations without opening the black box: Automated decisions and the gdpr, *Harv. JL & Tech.* 31 (2017) 841.
- [12] A.-K. Dombrowski, J. E. Gerken, K.-R. Müller, P. Kessel, Diffeomorphic counterfactuals with generative models, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46 (5) (2023) 3257–3274.

- [13] M. A. Prado-Romero, B. Prenkaj, G. Stilo, F. Giannotti, A survey on graph counterfactual explanations: Definitions, methods, evaluation, and research challenges, *ACM Comput. Surv.* 56 (7) (Apr. 2024). doi:10.1145/3618105.
URL <https://doi.org/10.1145/3618105>
- [14] A. A. Hansen, P. Pegios, A. Calissano, A. Feragen, Graph counterfactual explainable AI via latent space traversal, in: *Northern Lights Deep Learning Conference 2025*, 2024.
URL <https://openreview.net/forum?id=Pyqnc9eWhB>
- [15] C. Vignac, I. Krawczuk, A. Siraudin, B. Wang, V. Cevher, P. Frossard, Digress: Discrete denoising diffusion for graph generation, in: *The Eleventh International Conference on Learning Representations*, 2023.
URL <https://openreview.net/forum?id=UaAD-Nu86WX>
- [16] A. Lucic, M. A. Ter Hoeve, G. Tolomei, M. De Rijke, F. Silvestri, Cfgnnexplainer: Counterfactual explanations for graph neural networks, in: *International Conference on Artificial Intelligence and Statistics*, PMLR, 2022, pp. 4499–4511.
- [17] J. Chen, S. Wu, A. Gupta, Z. Ying, D4explainer: In-distribution explanations of graph neural network via discrete denoising diffusion, in: *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
URL <https://openreview.net/forum?id=GJtP1ZEzua>
- [18] J. Ma, R. Guo, S. Mishra, A. Zhang, J. Li, Clear: Generative counterfactual explanations on graphs, *Advances in neural information processing systems* 35 (2022) 25895–25907.
- [19] J. Ma, I. Takigawa, A. Yamamoto, C2explainer: Customizable mask-based counterfactual explanation for graph neural networks, in: *Proceedings of the 2025 ACM Conference on Fairness, Accountability, and Transparency, FAccT '25*, Association for Computing Machinery, New York, NY, USA, 2025, p. 137–149. doi:10.1145/3715275.3732012.
URL <https://doi.org/10.1145/3715275.3732012>
- [20] D. Bechtoldt, S. Bender, Graph diffusion counterfactual explanation (2025). *arXiv:2511.16287*.
URL <https://arxiv.org/abs/2511.16287>
- [21] M. Ninniri, M. Podda, D. Bacciu, Classifier-free graph diffusion for molecular property targeting, in: A. Bifet, J. Davis, T. Krilavičius, M. Kull, E. Ntoutsi, I. Žliobaitė (Eds.), *Machine Learning and Knowledge Discovery in Databases. Research Track*, Springer Nature Switzerland, Cham, 2024, pp. 318–335.
- [22] J. Ho, T. Salimans, Classifier-free diffusion guidance (2022). *arXiv:2207.12598*.
URL <https://arxiv.org/abs/2207.12598>

- [23] C. J. Maddison, D. Tarlow, T. Minka, A* sampling, in: Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, K. Weinberger (Eds.), *Advances in Neural Information Processing Systems*, Vol. 27, Curran Associates, Inc., 2014.
URL https://proceedings.neurips.cc/paper_files/paper/2014/file/937debc749f041eb5700df7211ac795c-Paper.pdf
- [24] J. Kazius, R. McGuire, R. Bursi, Derivation and Validation of Toxicophores for Mutagenicity Prediction, *Journal of Medicinal Chemistry* 48 (1) (2005) 312–320. doi:10.1021/jm040835a.
URL <https://doi.org/10.1021/jm040835a>
- [25] B. Sanchez-Lengeling, J. Wei, B. Lee, E. Reif, P. Wang, W. Qian, K. McCloskey, L. Colwell, A. Wiltchko, Evaluating attribution for graph neural networks, in: H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, H. Lin (Eds.), *Advances in Neural Information Processing Systems*, Vol. 33, Curran Associates, Inc., 2020, pp. 5898–5910.
URL https://proceedings.neurips.cc/paper_files/paper/2020/file/417fbbf2e9d5a28a855a11894b2e795a-Paper.pdf
- [26] C. Agarwal, O. Queen, H. Lakkaraju, M. Zitnik, Evaluating explainability for graph neural networks, *Scientific Data* 10 (1) (2023) 144. doi:10.1038/s41597-023-01974-x.
URL <https://doi.org/10.1038/s41597-023-01974-x>
- [27] K. M. Borgwardt, C. S. Ong, S. Schönauer, S. V. N. Vishwanathan, A. J. Smola, H.-P. Kriegel, Protein function prediction via graph kernels, *Bioinformatics* 21 (1) (2005) 47–56. doi:10.1093/bioinformatics/bti1007.
URL <https://doi.org/10.1093/bioinformatics/bti1007>
- [28] P. D. Dobson, A. J. Doig, Distinguishing enzyme structures from non-enzymes without alignments, *Journal of Molecular Biology* 330 (4) (2003) 771–783. doi:[https://doi.org/10.1016/S0022-2836\(03\)00628-4](https://doi.org/10.1016/S0022-2836(03)00628-4).
URL <https://www.sciencedirect.com/science/article/pii/S0022283603006284>
- [29] S. Pan, J. Wu, X. Zhu, C. Zhang, Graph ensemble boosting for imbalanced noisy graph stream classification, *IEEE Transactions on Cybernetics* 45 (5) (2015) 954–968. doi:10.1109/TCYB.2014.2341031.
- [30] S. Pan, J. Wu, X. Zhu, Cogboost: Boosting for fast cost-sensitive graph classification, *IEEE Transactions on Knowledge and Data Engineering* 27 (11) (2015) 2933–2946. doi:10.1109/TKDE.2015.2391115.
- [31] J. Tan, S. Geng, Z. Fu, Y. Ge, S. Xu, Y. Li, Y. Zhang, Learning and evaluating graph neural network explanations based on counterfactual and factual reasoning, in: *Proceedings of the ACM Web Conference 2022, WWW ’22*, Association for Computing Machinery, New York, NY, USA, 2022, p.

- 1018–1027. doi:10.1145/3485447.3511948.
URL <https://doi.org/10.1145/3485447.3511948>
- [32] M. D. Sanctis, R. D. Sanctis, S. Faralli, P. Velardi, B. Prenkaj, Beyond edge deletion: A comprehensive approach to counterfactual explanation in graph neural networks (2026). arXiv:2603.04209.
URL <https://arxiv.org/abs/2603.04209>
 - [33] F. Giorgi, F. Silvestri, G. Tolomei, Unified counterfactual explainer for graph neural networks, npj Artificial Intelligence (Jul. 2026). doi:10.1038/s44387-026-00135-w.
URL <https://doi.org/10.1038/s44387-026-00135-w>
 - [34] Z. Dehghanighobadi, A. Fischer, M. B. Zafar, Can llms explain themselves counterfactually?, in: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, 2025, pp. 7798–7826.
 - [35] W. Klos, S. Bender, L. Kades, Protein counterfactuals via diffusion-guided latent optimization, ICLR 2026 Gen² workshop (2026).
 - [36] C. Russell, Efficient search for diverse coherent explanations, in: Proceedings of the conference on fairness, accountability, and transparency, 2019, pp. 20–28.
 - [37] R. K. Mothilal, A. Sharma, C. Tan, Explaining machine learning classifiers through diverse counterfactual explanations, in: Proceedings of the 2020 conference on fairness, accountability, and transparency, 2020, pp. 607–617.
 - [38] P. Rodriguez, M. Caccia, A. Lacoste, L. Zamparo, I. Laradji, L. Charlin, D. Vazquez, Beyond trivial counterfactual explanations with diverse valuable explanations, in: ICCV, 2021, pp. 1056–1065.
 - [39] M. Augustin, V. Boreiko, F. Croce, M. Hein, Diffusion visual counterfactual explanations, NeurIPS 35 (2022) 364–377.
 - [40] G. Jeanneret, L. Simon, F. Jurie, Diffusion models for counterfactual explanations, in: Proceedings of the Asian conference on computer vision, 2022, pp. 858–876.
 - [41] N. Weng, P. Pegios, E. Petersen, A. Feragen, S. Bigdeli, Fast diffusion-based counterfactuals for shortcut removal and generation, in: ECCV, Springer, 2024, pp. 338–357.
 - [42] S. Bender, J. Herrmann, K.-R. Müller, G. Montavon, Towards desiderata-driven design of visual counterfactual explainers, in: Pattern Recognition, Vol. 174, 2026, p. 112811. doi:<https://doi.org/10.1016/j.patcog.2025.112811>.
 - [43] A. Zeid, S. Bender, Sce-lite-hq: Smooth visual counterfactual explanations with generative foundation models, arXiv preprint arXiv:2603.17048 (2026).

- [44] G. Jeanneret, L. Simon, F. Jurie, Adversarial counterfactual visual explanations, in: CVPR, 2023, pp. 16425–16435.
- [45] G. Jeanneret, L. Simon, f. Jurie, Text-to-image models for counterfactual explanations: a black-box approach, in: WACV, 2024, pp. 4757–4767.
- [46] S. Bender, M. Morik, Visual disentangled diffusion autoencoders: Scalable counterfactual generation for foundation models, ICLR 2026 Trustworthy AI workshop (2026).
- [47] S. Verma, B. Armgaan, S. Medya, S. Ranu, InduCE: Inductive counterfactual explanations for graph neural networks, Transactions on Machine Learning Research (2024).
URL <https://openreview.net/forum?id=RZPN8cgqST>
- [48] M. Kosan, Z. Huang, S. Medya, S. Ranu, A. Singh, Gcfexplainer: Global counterfactual explainer for graph neural networks, ACM Trans. Intell. Syst. Technol. 16 (5) (Aug. 2025). doi:10.1145/3698108.
URL <https://doi.org/10.1145/3698108>
- [49] M. A. Prado-Romero, B. Prenkaj, G. Stilo, Robust stochastic graph generator for counterfactual explanations, in: Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence, AAAI’24/IAAI’24/EAAI’24, AAAI Press, 2024. doi:10.1609/aaai.v38i19.30149.
URL <https://doi.org/10.1609/aaai.v38i19.30149>
- [50] D. Numeroso, D. Bacciu, Meg: Generating molecular counterfactual explanations for deep graph networks, in: 2021 International Joint Conference on Neural Networks (IJCNN), IEEE, 2021, pp. 1–8.
- [51] Y. He, Z. Zheng, P. Soga, Y. Zhu, Y. Dong, J. Li, Explaining graph neural networks with large language models: A counterfactual perspective on molecule graphs, in: Y. Al-Onaizan, M. Bansal, Y.-N. Chen (Eds.), Findings of the Association for Computational Linguistics: EMNLP 2024, Association for Computational Linguistics, Miami, Florida, USA, 2024, pp. 7079–7096. doi:10.18653/v1/2024.findings-emnlp.415.
URL <https://aclanthology.org/2024.findings-emnlp.415/>
- [52] L. S. Shapley, A value for n-person games, in: H. W. Kuhn, A. W. Tucker (Eds.), Contributions to the Theory of Games II, Princeton University Press, Princeton, 1953, pp. 307–317.
- [53] W. R. Swartout, J. D. Moore, Explanation in Second Generation Expert Systems, Springer Berlin Heidelberg, 1993, p. 543–585.

- [54] W. Samek, A. Binder, G. Montavon, S. Lapuschkin, K.-R. Müller, Evaluating the visualization of what a deep neural network has learned, *IEEE Trans. Neural Networks Learn. Syst.* 28 (11) (2017) 2660–2673.
- [55] M. Nauta, J. Trienes, S. Pathak, E. Nguyen, M. Peters, Y. Schmitt, J. Schlötterer, M. van Keulen, C. Seifert, From anecdotal evidence to quantitative evaluation methods: A systematic review on evaluating explainable AI, *ACM Comput. Surv.* 55 (13s) (2023) 295:1–295:42.
- [56] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, Z. Wojna, Rethinking the inception architecture for computer vision, in: *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 2818–2826.
- [57] D. Stutz, M. Hein, B. Schiele, Disentangling adversarial robustness and generalization, in: *CVPR, Computer Vision Foundation / IEEE*, 2019, pp. 6976–6987.
- [58] N. Yang, K. Zeng, Q. Wu, X. Jia, J. Yan, Learning substructure invariance for out-of-distribution molecular representations, in: S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, A. Oh (Eds.), *Advances in Neural Information Processing Systems*, Vol. 35, Curran Associates, Inc., 2022, pp. 12964–12978. doi:10.52202/068431-0942.
URL https://proceedings.neurips.cc/paper_files/paper/2022/file/547108084f0c2af39b956f8eadb75d1b-Paper-Conference.pdf
- [59] RDKit, online, Rdkit: Open-source cheminformatics, <https://www.rdkit.org>.
- [60] R. Huben, H. Cunningham, L. Smith, A. Ewart, L. Sharkey, Sparse autoencoders find highly interpretable features in language models, in: *International Conference on Learning Representations*, Vol. 2024, 2024, pp. 7827–7845.
- [61] J. Ho, A. Jain, P. Abbeel, Denoising diffusion probabilistic models, in: H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, H. Lin (Eds.), *Advances in Neural Information Processing Systems*, Vol. 33, Curran Associates, Inc., 2020, pp. 6840–6851.
URL https://proceedings.neurips.cc/paper_files/paper/2020/file/4c5bcfec8584af0d967f1ab10179ca4b-Paper.pdf
- [62] P. Dhariwal, A. Nichol, Diffusion models beat gans on image synthesis, *Advances in neural information processing systems* 34 (2021) 8780–8794.
- [63] I. Huberman-Spiegelglas, V. Kulikov, T. Michaeli, An edit friendly ddpm noise space: Inversion and manipulations, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 12469–12478.

- [64] C. Morris, N. M. Kriege, F. Bause, K. Kersting, P. Mutzel, M. Neumann, TUDataset: A collection of benchmark datasets for learning with graphs, in: ICML Workshop on Graph Representation Learning and Beyond (GRL+), 2020.
- [65] S. Lapuschkin, S. Wäldchen, A. Binder, G. Montavon, W. Samek, K.-R. Müller, Unmasking clever hans predictors and assessing what machines really learn, *Nature communications* 10 (1) (2019) 1096.
- [66] S. Bender, C. J. Anders, P. Chormai, H. A. Marxfeld, J. Herrmann, G. Montavon, Towards fixing clever-hans predictors with counterfactual knowledge distillation, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 2607–2615.
- [67] S. Bender, O. Delzer, J. Herrmann, H. A. Marxfeld, K.-R. Müller, G. Montavon, Mitigating clever hans strategies in image classifiers through generating counterexamples, *Information Fusion* (2026) 104406.

Appendix A. Full Algorithm

Algorithm 1: Graph Diffusion Counterfactual Explanation via Inversion (GDCE-I)

Input: Clean graph $G = (X, E, y)$, target y' , GNN classifier f , budget schedule $\mathcal{T}_{skip}$

Output: $G_{CE} = (X_{CE}, E_{CE}, y')$

// Phase 1: reference trajectory (states only)

```

1  $\hat{G}^0 \leftarrow G$ ;
2 for  $t = 1$  to  $T$  do
3    $\hat{X}^t \sim \hat{X}^{t-1} Q_X^t, \quad \hat{E}^t \sim \hat{E}^{t-1} Q_E^t$ ; // single-step forward, Eq. 6
4 end

// Phase 2: posterior Gumbel recording (reverse, original condition  $y$ )
5  $\mathcal{G} \leftarrow \emptyset$ ;
6 for  $s = T - 1$  to  $0$  do
7    $p_\theta(x_i^s | \hat{G}^{s+1}, y) \leftarrow \sum_x q(x_i^s | \hat{x}_i^{s+1}, x_i^0 = x) f_\theta(x_i^0 = x | \hat{G}^{s+1}, y)$ ;
8    $p_\theta(e_{ij}^s | \hat{G}^{s+1}, y) \leftarrow \sum_e q(e_{ij}^s | \hat{e}_{ij}^{s+1}, e_{ij}^0 = e) f_\theta(e_{ij}^0 = e | \hat{G}^{s+1}, y)$ ;
9    $g_s^X \leftarrow \text{POSTERIORGUMBEL}(\log p_\theta(X^s | \hat{G}^{s+1}, y), \hat{X}^s)$ ; // Eqs. 15, 16 [23]
10   $g_s^E \leftarrow \text{POSTERIORGUMBEL}(\log p_\theta(E^s | \hat{G}^{s+1}, y), \hat{E}^s)$ ;
11   $\mathcal{G} \leftarrow \mathcal{G} \cup \{(g_s^X, g_s^E)\}$ 
12 end

// Phase 3: dynamic budget search + guided replay (target condition  $y'$ )
13 for  $\tau \in \mathcal{T}_{skip}$  do
14    $G^\tau \leftarrow \hat{G}^\tau$ ;
15   for  $t = \tau$  to  $1$  do
16      $p_\theta(x_i^{t-1} | G^t, y') \leftarrow \sum_x q(x_i^{t-1} | x_i^t, x_i^0 = x) f_\theta(x_i^0 = x | G^t, y')$ ;
17      $p_\theta(e_{ij}^{t-1} | G^t, y') \leftarrow \sum_e q(e_{ij}^{t-1} | e_{ij}^t, e_{ij}^0 = e) f_\theta(e_{ij}^0 = e | G^t, y')$ ;
18      $X^{t-1} \leftarrow \text{one\_hot}(\text{argmax}(\log p_\theta(X^{t-1} | G^t, y') + g_{t-1}^X))$ ;
19      $E^{t-1} \leftarrow \text{one\_hot}(\text{argmax}(\log p_\theta(E^{t-1} | G^t, y') + g_{t-1}^E))$ ;
20      $G^{t-1} \leftarrow (X^{t-1}, E^{t-1})$ ;
21   end
22   if  $f(G^0) == y'$  then
23     return  $G_{CE} \leftarrow G^0$ ; // minimal counterfactual
24   end
25 end
26 return Failure; // no valid counterfactual within budgets

```

Appendix B. Datasets and Classifier Architectures

We evaluate on four graph-classification datasets, two molecular and two non-molecular. **Mutagenicity** [24] is a standard molecular benchmark from the TUDataset collection [64]. **Benzene** [25, 26] is a molecular attribution benchmark whose label indicates the presence of a benzene ring. **PROTEINS** [27]

and **TWITTER** (TWITTER-Real-Graph-Partial) [29] are likewise taken from the TUDataset collection. All four are preprocessed identically for GDCE-I and every baseline, so no method gains an advantage from a different data representation.

Graphs larger than 50 nodes are removed. We additionally apply the Huang frequency filter (node types occurring ≤ 50 times globally are dropped, together with the graphs containing them). On Benzene, this removes 36 graphs containing phosphorus. Mutagenicity and Benzene retain bond types as a 5-class edge encoding (0 = no bond, 1–4 = single/double/triple/aromatic), whereas PROTEINS and TWITTER carry no edge types and use a binary adjacency. Table B.5 summarizes the resulting statistics.

The two non-molecular datasets follow the same recipe. PROTEINS needs no vocabulary filter. Its nodes carry three secondary-structure labels (helix, sheet, turn), all of them frequent, and the 50-node cap leaves 871 of the 1113 graphs. TWITTER does need one. Its raw vocabulary of 1323 words would make the (b, n, d_x, d_x) attention tensor of the denoising network exceed device memory at the batch size we use, so we apply the same frequency rule as for the molecular sets with a threshold of 500 occurrences. This leaves 238 word types and 65031 graphs, retains 78.9% of all node instances and leaves the class balance essentially untouched. Nodes are words, edges word co-occurrences, and the binary label is the sentiment of the tweet, with class 0 positive and class 1 negative.

Dataset	Source	Graphs (tr/val/test)	Node types	Edge classes	Max nodes	Balance (0/1)
Mutagenicity	TUDataset [64, 24]	2377/792/792 (3961)	13	5	50	57.6/42.4
Benzene	[25, 26]	7178/2393/2393 (11964)	8	5	25	49.9/50.1
PROTEINS	TUDataset [64, 27]	523/174/174 (871)	3	2	50	52.9/47.1
TWITTER	TUDataset [64, 29]	39019/13006/13006 (65031)	238	2	50	52.2/47.8

Table B.5: Dataset statistics after preprocessing.

SMILES validity ceiling on Benzene. Because Benzene molecules are capped at 25 nodes in the source data, some aromatic rings are truncated at the boundary. RDKit cannot sanitize a partial aromatic system, so even a perfectly reconstructed graph may fail to yield a valid SMILES string. Only 89.05% pass strict sanitization. This is an upper bound on the SMILES column of Table 2 for Benzene. The reported values should be read against 0.8905 rather than 1.0. For comparison, the ceiling on Mutagenicity is 0.976.

The shared classifiers. Every method in Table 2 is optimized against, and evaluated by, the same classifier per dataset. We train two per dataset on the shared training split: a binary-adjacency **GCN** (LEConv, 3×128 hidden, BatchNorm, dropout 0.3, mean pooling), which every method can consume and against which all of them are measured, and an edge-feature-aware **GINE** (GINEConv with a 5-class edge encoder, otherwise identical), reserved for the two methods that predict bond types (GDCE-I and UCExplainer). PROTEINS and TWITTER carry no edge types and therefore have a GCN only. Both are plain PyTorch modules with no framework dependency, so the identical weights are loaded

inside each baseline’s own container. We verified that all five repositories reproduce the same predictions on the same graphs before running any evaluation. Test accuracies: Mutagenicity 0.806 (GCN) / 0.813 (GINE), Benzene 0.919 (GCN) / 1.000 (GINE), PROTEINS 0.632 (GCN), TWITTER 0.648 (GCN).

Because every method is trained and scored against these classifiers, class labels are taken from the classifier rather than from the dataset annotation, so that “flipping the label” and “flipping the classifier” coincide by construction.